

Les bases



FORMATION C# - LES BASES

C#

Ecrire des applications modernes et
typees, pas a pas.

DanielCraft - Livre debutant - Pack Web

Sommaire

Clique un chapitre ou un sous-chapitre pour y aller.

1. [C'est quoi C# ?](#)

- [Le langage en une phrase](#)
- [Pourquoi apprendre C# ?](#)
- [A quoi ca ressemble ?](#)
- [Petite histoire](#)
- [En vrai](#)
- [A retenir](#)

2. [Installer C#](#)

- [Ce qu'il te faut](#)
- [Installer le SDK .NET](#)
- [Creer un premier projet](#)
- [Installer VS Code](#)
- [Petite histoire](#)
- [A retenir](#)

3. [Premier programme](#)

- [Hello World moderne](#)
- [La version classique](#)
- [Afficher plusieurs lignes](#)
- [Les commentaires](#)
- [Petite histoire](#)
- [A retenir](#)

4. [Les variables](#)

- [C'est quoi une variable ?](#)
- [Regles de nommage](#)
- [Modifier une variable](#)
- [Interpolation de chaines](#)
- [Constantes](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [A retenir](#)

5. [Les types de donnees](#)

- [Les types de base](#)
- [Types valeur vs reference](#)
- [Conversion](#)
- [Operations](#)
- [Petite histoire](#)
- [A retenir](#)

6. [Les conditions](#)

- [Prendre une decision](#)
- [if / else if / else](#)

- [Operateurs de comparaison](#)
- [Operateurs logiques](#)
- [Switch](#)
- [Petite histoire](#)
- [A retenir](#)

7. [Les boucles](#)

- [Repeter du code](#)
- [La boucle for](#)
- [La boucle foreach](#)
- [La boucle while](#)
- [break et continue](#)
- [Petite histoire](#)
- [A retenir](#)

8. [Les methodes](#)

- [Pourquoi des methodes ?](#)
- [Retourner une valeur](#)
- [Parametres optionnels](#)
- [Expression-bodied](#)
- [Petite histoire](#)
- [A retenir](#)

9. [Tableaux et listes](#)

- [Les tableaux \(Array\)](#)
- [Parcourir un tableau](#)
- [Les listes \(List<T>\)](#)
- [Methodes utiles de List](#)
- [Petite histoire](#)
- [A retenir](#)

10. [Classes et objets](#)

- [C'est quoi une classe ?](#)
- [Constructeur](#)
- [Encapsulation](#)
- [Petite histoire](#)
- [A retenir](#)

11. [L'heritage](#)

- [Reutiliser du code](#)
- [virtual et override](#)
- [Le mot-cle base](#)
- [Classes abstraites](#)
- [Petite histoire](#)
- [A retenir](#)

12. [Gerer les erreurs](#)

- [Les exceptions](#)
- [Les exceptions courantes](#)

- [finally](#)
 - [Lever une exception](#)
 - [Petite histoire](#)
 - [A retenir](#)
13. [Mini-projet : gestionnaire de taches](#)
- [L'objectif](#)
 - [Structure](#)
 - [Ce que tu apprends](#)
 - [A retenir](#)
14. [Ce qu'il faut retenir](#)
- [Les briques essentielles](#)
 - [La methode](#)
 - [Ce qui vient apres](#)
 - [A retenir](#)
15. [Atelier : variables et types](#)
- [Objectif](#)
 - [Exercice 1 : carte d'identite](#)
 - [Exercice 2 : convertisseur EUR -> USD](#)
 - [Exercice 3 : temperature](#)
 - [Defi : aire et perimetre](#)
 - [A retenir](#)
16. [Atelier : methodes](#)
- [Objectif](#)
 - [Exercice 1 : salutation](#)
 - [Exercice 2 : moyenne](#)
 - [Exercice 3 : mot de passe valide](#)
 - [Exercice 4 : factorielle](#)
 - [A retenir](#)
17. [Atelier : classes](#)
- [Objectif](#)
 - [Exercice 1 : classe Produit](#)
 - [Exercice 2 : panier d'achat](#)
 - [Exercice 3 : heritage Vehicule](#)
 - [A retenir](#)
18. [Erreurs classiques en C#](#)
- [Les pieges des debutants](#)
 - [1. NullReferenceException](#)
 - [2. Oublier break dans switch](#)
 - [3. Comparer des strings avec ==](#)
 - [4. Modifier une collection en iterant](#)
 - [5. Index hors limites](#)
 - [6. Oublier le point-virgule](#)
 - [7. Confondre = et ==](#)

- [A retenir](#)

19. [Bonnes pratiques](#)

- [Conventions de nommage](#)
- [Structure d'un fichier](#)
- [Principes](#)
- [Documentation](#)
- [A retenir](#)

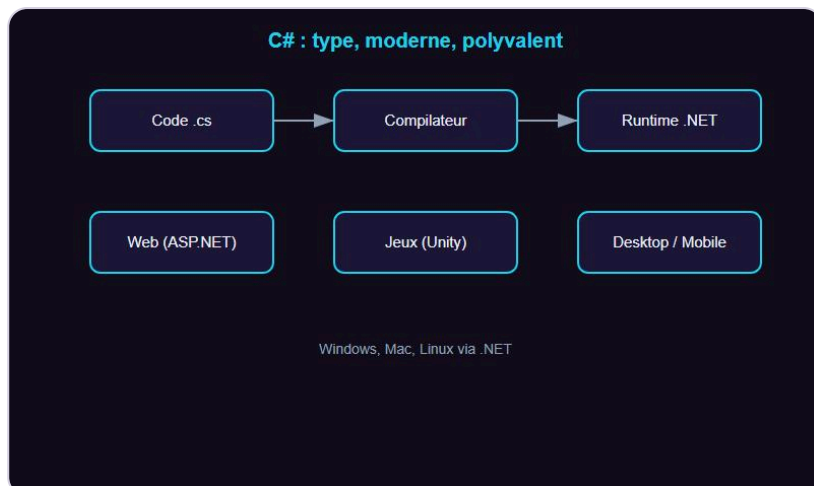
20. [Quiz C# - Les bases](#)

- [Teste tes connaissances](#)
- [Reponses](#)

21. [Bravo !](#)

- [Tu as termine C# - Les bases](#)
- [Ce que tu as construit](#)
- [La suite avec DanielCraft](#)
- [Un dernier conseil](#)

C'est quoi C# ?



C# : type, moderne, polyvalent.

Le langage en une phrase

C# (prononce "C sharp") est un langage cree par Microsoft en 2000. Il sert a developper des applications Windows, des jeux video (Unity), des API web et des applications mobiles. Sa syntaxe est claire, typee et moderne.

Pourquoi apprendre C# ?

C# est dans le top 5 mondial (TIOBE, Stack Overflow). Il est utilise par des millions de developpeurs pour des projets professionnels. Unity, le moteur de jeux le plus populaire, utilise C# comme langage principal.

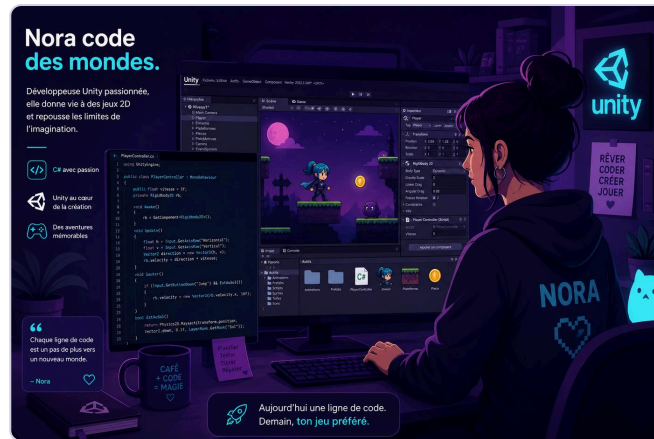
> **Astuce DanielCraft** - Si tu veux creer des jeux ou travailler dans l'ecosysteme Microsoft, C# est le choix evident.

Nora decouvre C# en cherchant comment creer un jeu 2D. Elle installe Visual Studio, tape quelques lignes et voit un personnage bouger a l'ecran.

A quoi ca ressemble ?

```
string nom = "Lea";
Console.WriteLine($"Salut {nom}, bienvenue en C# !");
```

Ce code affiche : Salut Lea, bienvenue en C# !



Nora découvre C# avec Unity.

Petite histoire

Anders Hejlsberg, déjà créateur de Turbo Pascal et Delphi, conçoit C# chez Microsoft. Le langage évolue vite : chaque version apporte des simplifications. Aujourd'hui C# 12+ est moderne, concis et performant.

En vrai

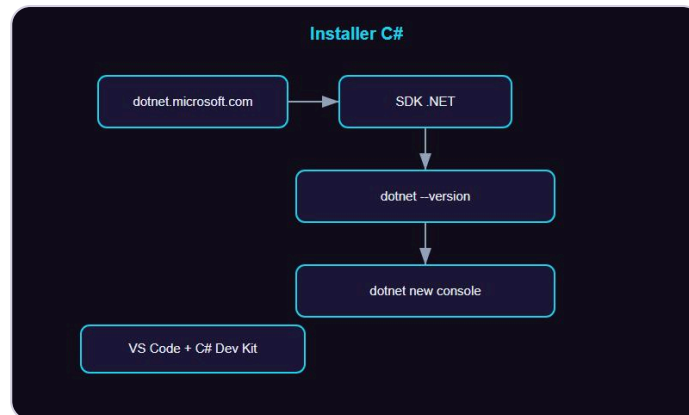
Sam utilise C# pour créer une API REST avec ASP.NET. Max développe un jeu de plateforme avec Unity. Nora automatise des tâches bureautiques avec des scripts C#.

> **Piège** - C# ressemble à Java mais ce n'est pas Java. Les conventions, l'écosystème et les outils diffèrent.

A retenir

- C# est type, moderne, polyvalent.
- Il tourne sur Windows, Mac, Linux via .NET.
- Jeux (Unity), web (ASP.NET), desktop, mobile (MAUI).

Installer C#



SDK .NET + VS Code : le duo de depart.

Ce qu'il te faut

Pour coder en C# tu as besoin du SDK .NET (gratuit) et d'un editeur. On recommande Visual Studio Code avec l'extension C# Dev Kit, ou Visual Studio Community (gratuit, Windows/Mac).

Installer le SDK .NET

1. Va sur dotnet.microsoft.com, telecharge le SDK .NET (derniere version LTS).
2. Lance l'installateur.
3. Ouvre un terminal et tape `dotnet --version`.
4. Si un numero de version s'affiche, c'est bon.

> **Astuce DanielCraft** - Le SDK inclut le compilateur et le runtime. Un seul telechargement suffit.

Creer un premier projet

```
dotnet new console -n MonPremierProjet
cd MonPremierProjet
dotnet run
```

Tu verras "Hello, World!" dans le terminal.

Installer VS Code

Telecharge VS Code sur code.visualstudio.com. Installe l'extension "C# Dev Kit" de Microsoft. Elle apporte IntelliSense, le debug et la gestion de projet.

Petite histoire

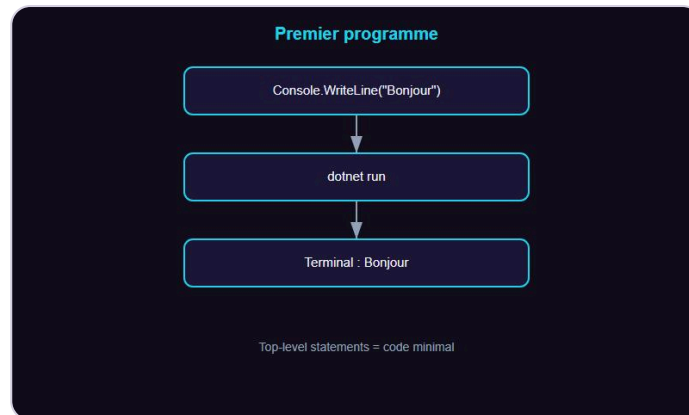
Max installe .NET en 3 minutes. Il tape `dotnet new console` et son premier programme fonctionne immediatement. Pas de configuration complexe.

A retenir

- Telecharge le SDK .NET sur dotnet.microsoft.com.
- Verifie avec `dotnet --version`.

- `dotnet new console` cree un projet pret a l'emploi.
- VS Code + C# Dev Kit = environnement leger et efficace.

Premier programme



`Console.WriteLine` : ta premiere ligne.

Hello World moderne

Depuis C# 10, un programme minimal tient en une ligne :

```
Console.WriteLine("Bonjour le monde !");
```

Cree un projet avec `dotnet new console`, remplace le contenu de `Program.cs`, puis `dotnet run`.

La version classique

```
namespace MonApp;

class Program
{
    static void Main(string[] args)
    {
        Console.WriteLine("Bonjour le monde !");
    }
}
```

Les deux versions produisent le meme resultat. La version courte utilise les "top-level statements".

> **Astuce DanielCraft** - Commence avec la version courte. Tu comprendras les namespaces et classes plus tard.

Afficher plusieurs lignes

```
Console.WriteLine("Ligne 1");
Console.WriteLine("Ligne 2");
Console.Write("Sans retour ");
Console.Write("a la ligne");
```

`WriteLine` ajoute un retour a la ligne, `Write` non.

Les commentaires

```
// Commentaire sur une ligne
/* Commentaire
   sur plusieurs lignes */
Console.WriteLine("Code execute");
```

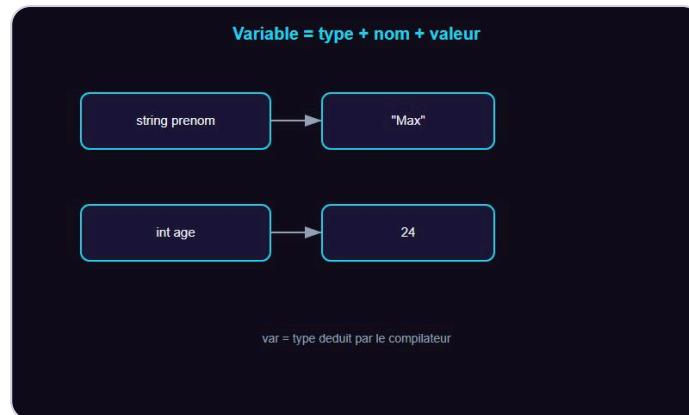
Petite histoire

Nora cree son premier projet. Elle modifie le message, relance `dotnet run`, et voit son texte. Premier programme en 2 minutes.

A retenir

- `Console.WriteLine()` affiche du texte.
- `dotnet run` compile et execute.
- Top-level statements = code minimal.

Les variables



Variable = type + nom + valeur.

C'est quoi une variable ?

Une variable est un espace memoire nomme qui contient une valeur. En C#, on declare le type ou on laisse le compilateur le deviner avec `var`.

```

string prenom = "Max";
int age = 24;
var ville = "Lyon"; // Le compilateur deduit string
  
```

Regles de nommage

- camelCase pour les variables locales : `monScore`, `prixTotal`.
- Pas de mots reserves (`int`, `class`, `if` ...).
- Commence par une lettre ou underscore.
- Sensible a la casse : `nom` et `Nom` sont differents.

> **Astuce DanielCraft** - Utilise `var` quand le type est evident. Explicit quand ca aide a la lisibilite.

Modifier une variable

```

int score = 0;
score = score + 10;
score += 5;
Console.WriteLine(score); // 15
  
```

Interpolation de chaines

```

string ville = "Paris";
Console.WriteLine($"Je vis a {ville}");
  
```

Le `$` avant les guillemets active l'interpolation (comme f-string en Python).

Constantes

```
const double TVA = 0.20;  
// TVA = 0.25; // Erreur ! Une constante ne change pas.
```

Petite histoire

Sam stocke `salaire = 1800` et `loyer = 650`. Il affiche `reste = salaire - loyer`. C# lui repond 1150, sans surprise.

Erreur classique

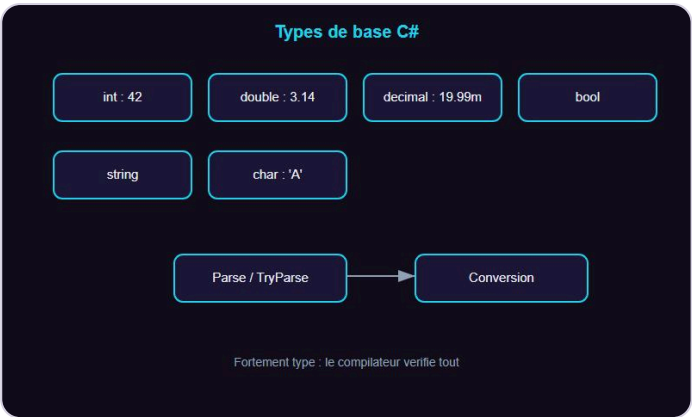
```
Console.WriteLine(prnom); // Erreur CS0103 : variable inexistante
```

Le compilateur refuse immédiatement. Pas d'attente jusqu'a l'execution.

A retenir

- Declarer avec un type ou `var`.
- camelCase pour les variables locales.
- `$"..."` pour l'interpolation.
- `const` pour les valeurs qui ne changent pas.

Les types de donnees



int, double, decimal, string, bool.

Les types de base

Type	Exemple	Usage
int	42	Entier
long	9000000000L	Grand entier
double	3.14	Decimal (default)
decimal	19.99m	Precision financiere
bool	true / false	Vrai ou faux
char	'A'	Un caractere
string	"Bonjour"	Texte

Types valeur vs reference

Les types simples (`int` , `double` , `bool`) sont des types valeur : la variable contient directement la donnee. Les `string` et objets sont des types reference : la variable pointe vers la donnee.

Conversion

```
string texte = "42";
int nombre = int.Parse(texte);           // Peut lever une exception
bool ok = int.TryParse(texte, out int n); // Plus sur
```

> **Astuce DanielCraft** - Prefere `TryParse` a `Parse` pour eviter les plantages sur des saisies invalides.

Operations

```
int a = 10, b = 3;
Console.WriteLine(a + b); // 13
Console.WriteLine(a / b); // 3 (division entiere)
Console.WriteLine(a % b); // 1 (reste)
Console.WriteLine((double)a / b); // 3.333...
```

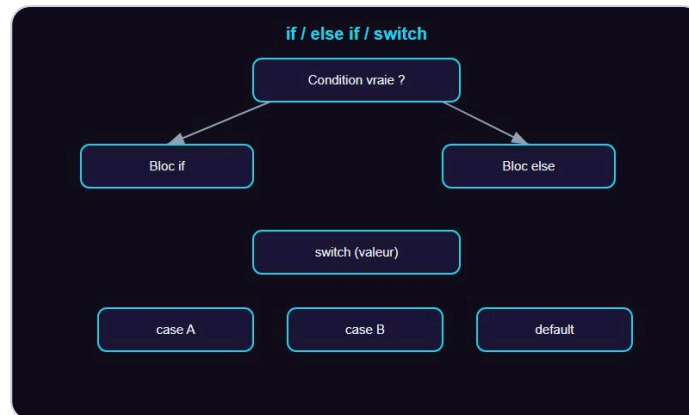
Petite histoire

Nora calcule un prix TTC avec `decimal` pour eviter les erreurs d'arrondi. `19.99m * 1.20m` donne exactement `23.988m`.

A retenir

- C# est fortement type : le compilateur verifie tout.
- `int`, `double`, `string`, `bool` pour 90% des cas.
- `decimal` pour l'argent.
- `TryParse` pour convertir sans risque.

Les conditions



if / else if / switch.

Prendre une décision

```

int age = 17;
if (age >= 18)
{
    Console.WriteLine("Majeur");
}
else
{
    Console.WriteLine("Mineur");
}
  
```

if / else if / else

```

int note = 14;
if (note >= 16)
    Console.WriteLine("Tres bien");
else if (note >= 12)
    Console.WriteLine("Bien");
else if (note >= 10)
    Console.WriteLine("Passable");
else
    Console.WriteLine("Insuffisant");
  
```

> **Astuce DanielCraft** - Les accolades sont facultatives pour une seule instruction, mais recommandées pour la lisibilité.

Operateurs de comparaison

Operateur	Signification
<code>==</code>	Egal
<code>!=</code>	Different
<code><</code> <code>></code>	Inferieur / superieur
<code><=</code> <code>>=</code>	Inferieur ou egal / superieur ou egal

Operateurs logiques

```
if (age >= 18 && permis)
    Console.WriteLine("Peut conduire");

if (estEtudiant || estSenior)
    Console.WriteLine("Tarif reduit");
```

Switch

```
string jour = "lundi";
switch (jour)
{
    case "lundi":
    case "mardi":
        Console.WriteLine("Debut de semaine");
        break;
    case "vendredi":
        Console.WriteLine("Presque le weekend");
        break;
    default:
        Console.WriteLine("Autre jour");
        break;
}
```

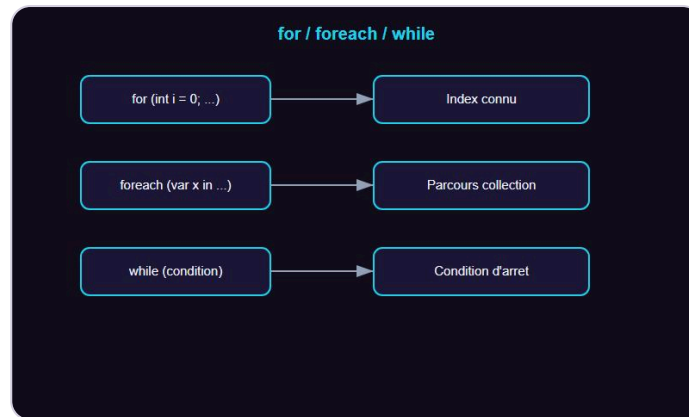
Petite histoire

Max écrit un programme qui vérifie l'âge et le permis avant d'afficher "Accès autorisé". Deux conditions, un `&&`, et c'est fait.

A retenir

- `if` / `else if` / `else` pour décider.
- `switch` pour comparer une valeur à plusieurs cas.
- `&&` (et), `||` (ou), `!` (non).

Les boucles



for / foreach / while.

Repeter du code

Une boucle execute un bloc plusieurs fois. C# propose `for`, `while`, `do-while` et `foreach`.

La boucle for

```
for (int i = 0; i < 5; i++)
{
    Console.WriteLine(i);
}
// Affiche 0, 1, 2, 3, 4
```

La boucle foreach

```
string[] fruits = { "pomme", "banane", "cerise" };
foreach (string fruit in fruits)
{
    Console.WriteLine(fruit);
}
```

`foreach` parcourt chaque element sans index.

La boucle while

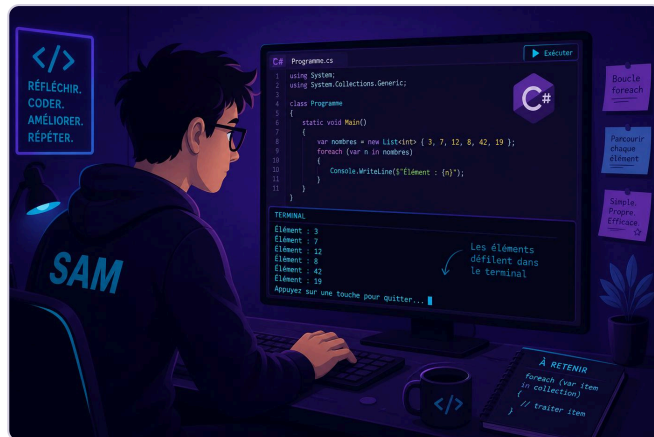
```
int compteur = 0;
while (compteur < 3)
{
    Console.WriteLine(compteur);
    compteur++;
}
```

> **Piege** - Oublier `compteur++` = boucle infinie. Ctrl+C pour arreter.

break et continue

```
for (int i = 0; i < 10; i++)  
{  
    if (i == 5) break;  
    if (i % 2 == 0) continue;  
    Console.WriteLine(i); // 1, 3  
}
```

> **Astuce DanielCraft** - `foreach` quand tu parcoures une collection. `for` quand tu as besoin de l'index.



Sam parcourt une collection.

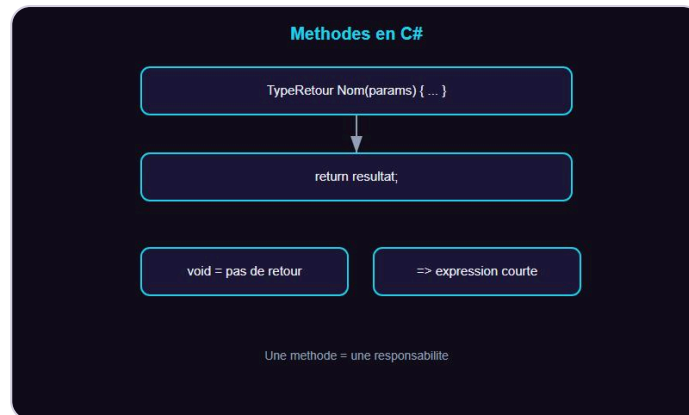
Petite histoire

Sam affiche les tables de multiplication avec deux boucles imbriquées. 4 lignes pour un résultat que recopier prendrait des heures.

A retenir

- `for` : nombre d'iterations connu.
- `foreach` : parcourir une collection.
- `while` : condition d'arrêt.
- `break` sort, `continue` passe au suivant.

Les methodes



Methode : type retour + nom + params.

Pourquoi des methodes ?

Une methode regroupe du code reutilisable sous un nom. En C# tout code vit dans une methode (meme implicitement avec les top-level statements).

```

void Saluer(string prenom)
{
    Console.WriteLine($"Bonjour {prenom} !");
}

Saluer("Nora");
Saluer("Max");
  
```

Retourner une valeur

```

int Additionner(int a, int b)
{
    return a + b;
}

int resultat = Additionner(3, 7);
Console.WriteLine(resultat); // 10
  
```

Parametres optionnels

```

void Presenter(string nom, string langue = "français")
{
    Console.WriteLine($"{nom} parle {langue}");
}

Presenter("Lea");           // Lea parle français
Presenter("Tom", "anglais"); // Tom parle anglais
  
```

> **Astuce DanielCraft** - Une methode = une responsabilite. Si elle fait trop de choses, decoupe-la.

Expression-bodied

```
double CalculerTtc(double ht) => ht * 1.20;
```

Pour les methodes courtes, la fleche `=>` remplace les accolades et le `return`.

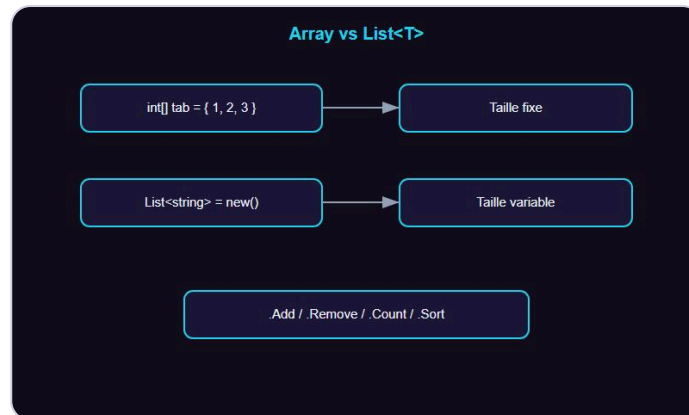
Petite histoire

Max ecrit 3 fois le meme calcul de TVA. Sam lui montre `CalculerTtc()`. Le code passe de 15 lignes a 5.

A retenir

- `void` si pas de retour, sinon le type retourne.
- Parametres optionnels avec `= valeur`.
- `=>` pour les methodes d'une expression.
- Une methode = une responsabilite.

Tableaux et listes



Array fixe vs List<T> flexible.

Les tableaux (Array)

Un tableau a une taille fixe, définie à la création.

```
int[] notes = { 14, 17, 11, 19, 8 };
Console.WriteLine(notes[0]); // 14
Console.WriteLine(notes.Length); // 5
```

Parcourir un tableau

```
foreach (int note in notes)
{
    Console.WriteLine(note);
}
```

Les listes (List<T>)

Une liste est redimensionnable. C'est le conteneur le plus utilisé.

```
List<string> fruits = new() { "pomme", "banane" };
fruits.Add("cerise");
fruits.Remove("banane");
Console.WriteLine(fruits.Count); // 2
```

Methodes utiles de List

Methodes	Role
<code>.Add(x)</code>	Ajoute a la fin
<code>.Remove(x)</code>	Supprime par valeur
<code>.Contains(x)</code>	Verifie la presence
<code>.Sort()</code>	Trie
<code>.Count</code>	Nombre d'elements

> **Astuce DanielCraft** - Utilise `List<T>` par default. Les tableaux sont utiles quand la taille est connue et fixe.

Petite histoire

Nora stocke les prenomms de sa classe dans une `List<string>`. Elle trie avec `.Sort()` et affiche avec `foreach`.

A retenir

- Tableau = taille fixe, acces par index.
- `List<T>` = taille variable, methodes pratiques.
- `foreach` pour parcourir sans index.

Classes et objets



Classe = plan. Objet = instance.

C'est quoi une classe ?

Une classe est un plan. Un objet est une instance de ce plan. La classe définit les propriétés et méthodes ; l'objet les utilise.

```

class Animal
{
    public string Nom { get; set; }
    public int Age { get; set; }

    public void SePresenter()
    {
        Console.WriteLine($"Je suis {Nom}, {Age} ans.");
    }
}

var chat = new Animal { Nom = "Felix", Age = 3 };
chat.SePresenter(); // Je suis Felix, 3 ans.
  
```

Constructeur

```

class Personne
{
    public string Nom { get; }
    public int Age { get; }

    public Personne(string nom, int age)
    {
        Nom = nom;
        Age = age;
    }
}

var p = new Personne("Lea", 28);
  
```

> **Astuce DanielCraft** - PascalCase pour les classes et propriétés. camelCase pour les variables locales.

Encapsulation

```
class CompteBancaire
{
    public decimal Solde { get; private set; }

    public void Depositer(decimal montant)
    {
        if (montant > 0) Solde += montant;
    }
}
```

`private set` empeche la modification directe depuis l'exterieur.

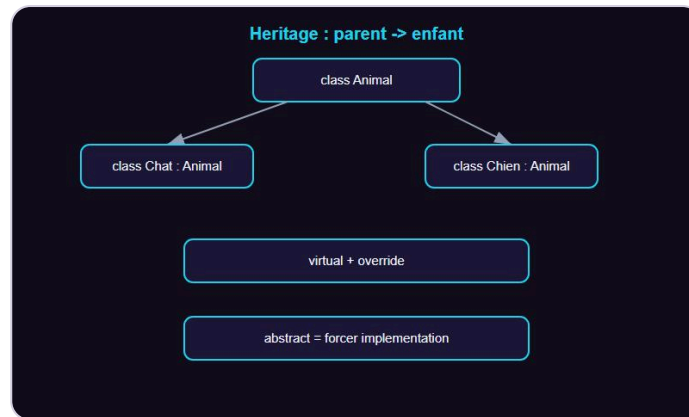
Petite histoire

Max modele un `Produit` avec un nom et un prix. Il cree une liste de produits et calcule le total avec LINQ (qu'il decouvrira au niveau intermediaire).

A retenir

- Classe = plan. Objet = exemplaire.
- Proprietes avec `{ get; set; }`.
- Constructeur pour initialiser.
- Encapsulation : controler l'accès aux données.

L'héritage



Heritage : parent -> enfant.

Reutiliser du code

L'héritage permet a une classe enfant de reprendre les membres d'une classe parent et d'en ajouter ou modifier.

```

class Animal
{
    public string Nom { get; set; }
    public virtual void Crier()
    {
        Console.WriteLine("...");
    }
}

class Chat : Animal
{
    public override void Crier()
    {
        Console.WriteLine("Miaou !");
    }
}

var felix = new Chat { Nom = "Felix" };
felix.Crier(); // Miaou !
  
```

virtual et override

- **virtual** dans le parent : autorise la redefinition.
- **override** dans l'enfant : remplace le comportement.

Le mot-cle base

```
class Chien : Animal
{
    public override void Crier()
    {
        base.Crier(); // Appelle la version parent
        Console.WriteLine("Wouf !");
    }
}
```

> **Astuce DanielCraft** - N'abuse pas de l'heritage. Prefere la composition quand la relation "est un" n'est pas claire.

Classes abstraites

```
abstract class Forme
{
    public abstract double Aire();
}

class Cercle : Forme
{
    public double Rayon { get; set; }
    public override double Aire() => Math.PI * Rayon * Rayon;
}
```

Une classe abstraite ne peut pas etre instanciee directement.

Petite histoire

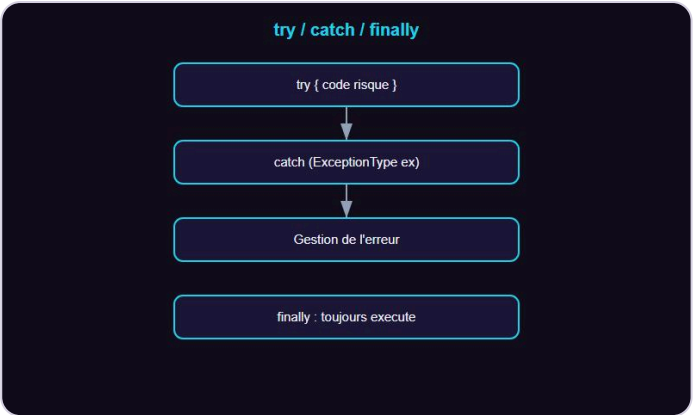
Nora cree une hierarchie **Vehicule** -> **Voiture** / **Moto** . Chaque sous-classe a ses propres regles de consommation.

A retenir

- **:** **ParentClass** pour heriter.
- **virtual** + **override** pour le polymorphisme.
- **abstract** pour forcer l'implementation.
- Composition > heritage quand le doute existe.

CHAPITRE 12

Gerer les erreurs



try/catch/finally.

Les exceptions

En C#, les erreurs a l'execution sont des exceptions. On les attrape avec `try/catch`.

```
try
{
    int nombre = int.Parse(Console.ReadLine());
    Console.WriteLine(10 / nombre);
}
catch (FormatException)
{
    Console.WriteLine("Ce n'est pas un nombre.");
}
catch (DivideByZeroException)
{
    Console.WriteLine("Division par zero impossible.");
}
```

Les exceptions courantes

Exception	Cause
<code>NullReferenceException</code>	Acces a un objet null
<code>IndexOutOfRangeException</code>	Index hors limites
<code>FormatException</code>	Conversion invalide
<code>DivideByZeroException</code>	Division par zero
<code>FileNotFoundException</code>	Fichier introuvable
<code>InvalidOperationException</code>	Operation non valide

finally

```
try
{
    // Code risque
}
catch (Exception ex)
{
    Console.WriteLine($"Erreur : {ex.Message}");
}
finally
{
    Console.WriteLine("Toujours execute");
}
```

Lever une exception

```
void Retirer(decimal solde, decimal montant)
{
    if (montant > solde)
        throw new InvalidOperationException("Solde insuffisant");
}
```

> **Astuce DanielCraft** - N'attrape que ce que tu sais gerer. Laisser remonter aide au debug.



Max lit la NullReferenceException.

Petite histoire

Max demande un nombre. L'utilisateur tape "abc". Sans `try/catch` le programme plante. Avec, il redemande poliment.

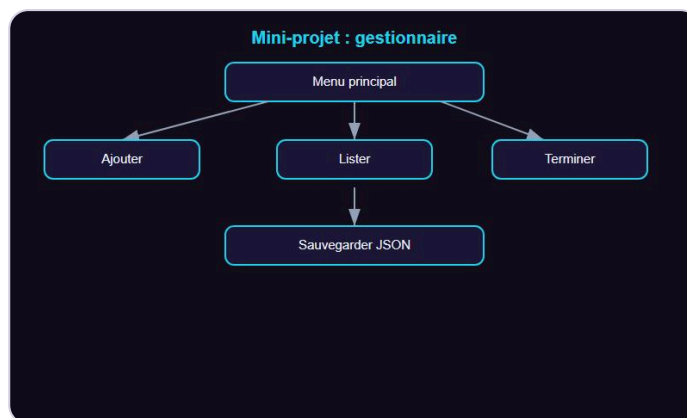
A retenir

- `try/catch` pour attraper les exceptions.
- Précise le type d'exception.
- `throw` pour lever tes propres erreurs.
- `finally` s'exécute toujours.

Mini-projet : gestionnaire de tâches



Nora, Max et Sam valident le projet.



Mini-projet : gestionnaire de tâches.

L'objectif

On crée un programme console qui permet d'ajouter, lister, marquer comme fait et supprimer des tâches. Ce projet combine classes, listes, méthodes, boucles, conditions et fichiers.

Structure

```

using System.Text.Json;

var taches = Charger();
bool quitter = false;

while (!quitter)
{
    Console.WriteLine("\n1. Ajouter 2. Lister 3. Terminer 4. Supprimer 5. Quitter");
    switch (Console.ReadLine())
    {
        case "1": Ajouter(); break;
        case "2": Lister(); break;
        case "3": Terminer(); break;
        case "4": Supprimer(); break;
        case "5": quitter = true; break;
    }
}
  
```

```

}

void Ajouter()
{
    Console.Write("Tache : ");
    string titre = Console.ReadLine() ?? "";
    taches.Add(new Tache(titre));
    Sauvegarder();
    Console.WriteLine("Ajoutée.");
}

void Lister()
{
    if (taches.Count == 0) { Console.WriteLine("Aucune tache."); return; }
    for (int i = 0; i < taches.Count; i++)
    {
        var t = taches[i];
        string statut = t.Fait ? "[x]" : "[ ]";
        Console.WriteLine($" {i + 1}. {statut} {t.Titre}");
    }
}

void Terminer()
{
    Console.Write("Numero : ");
    if (int.TryParse(Console.ReadLine(), out int n) && n > 0 && n <= taches.Count)
    {
        taches[n - 1].Fait = true;
        Sauvegarder();
    }
}

void Supprimer()
{
    Console.Write("Numero : ");
    if (int.TryParse(Console.ReadLine(), out int n) && n > 0 && n <= taches.Count)
    {
        taches.RemoveAt(n - 1);
        Sauvegarder();
    }
}

List<Tache> Charger()
{
    if (!File.Exists("taches.json")) return new();
    string json = File.ReadAllText("taches.json");
    return JsonSerializer.Deserialize<List<Tache>>(json) ?? new();
}

void Sauvegarder()
{
    string json = JsonSerializer.Serialize(taches, new JsonSerializerOptions { WriteIndented = true });
    File.WriteAllText("taches.json", json);
}

class Tache
{
    public string Titre { get; set; } = "";
    public bool Fait { get; set; }
    public Tache() { }
    public Tache(string titre) => Titre = titre;
}

```

Ce que tu apprends

- `System.Text.Json` pour la persistance.
- Top-level statements avec methodes locales.
- `List<T>` et manipulation d'index.
- Boucle principale avec menu.

> **Astuce DanielCraft** - Commence par le squelette (menu + methodes vides), puis remplis une a une.

A retenir

- Un projet = assemblage de notions.
- Decouper en methodes rend le code lisible.
- JSON pour sauvegarder des donnees structurees.

Ce qu'il faut retenir



La carte des notions C#.

Les briques essentielles

Concept	Resume
Variables	Type explicite ou <code>var</code> , camelCase
Types	<code>int</code> , <code>double</code> , <code>string</code> , <code>bool</code> , <code>decimal</code>
Conditions	<code>if / else if / else</code> , <code>switch</code>
Boucles	<code>for</code> , <code>foreach</code> , <code>while</code>
Methodes	Type retour + nom + params, <code>=></code>
Tableaux/Listes	Array fixe, <code>List<T></code> flexible
Classes	Proprietes, constructeur, encapsulation
Heritage	<code>: Parent</code> , <code>virtual / override</code>
Erreurs	<code>try/catch/finally</code> , <code>throw</code>

La methode

1. Lis le chapitre.
 2. Tape les exemples (pas de copier-coller).
 3. Modifie pour voir ce qui change.
 4. Fais les ateliers.
 5. Reviens quand tu bloques.
- > **Astuce DanielCraft** - Le compilateur C# est ton ami. Chaque erreur est une indication precise.

Ce qui vient apres

- LINQ et les requetes sur collections.

- Async/await et la programmation asynchrone.
- Les interfaces et l'injection de dependances.
- ASP.NET pour le web, Unity pour les jeux.

A retenir

- Tu as les bases. Le compilateur te guide.
- Chaque projet termine renforce ta confiance.

Atelier : variables et types



Atelier : variables et types.

Objectif

Pratiquer les declarations, conversions et interpolations.

Exercice 1 : carte d'identite

```
string prenom = "Sam";
int age = 22;
string ville = "Nantes";
Console.WriteLine($"Je suis {prenom}, {age} ans, je vis a {ville}.");
```

Exercice 2 : convertisseur EUR -> USD

```
Console.Write("Montant en EUR : ");
decimal euros = decimal.Parse(Console.ReadLine());
decimal dollars = euros * 1.08m;
Console.WriteLine($"{euros} EUR = {dollars:F2} USD");
```

Exercice 3 : temperature

Convertir Celsius en Fahrenheit : $F = C * 9/5 + 32$.

```
Console.Write("Celsius : ");
double c = double.Parse(Console.ReadLine());
double f = c * 9.0 / 5.0 + 32.0;
Console.WriteLine($"{c}°C = {f:F1}°F");
```

Defi : aire et perimetre

```
Console.Write("Rayon : ");
double rayon = double.Parse(Console.ReadLine());
Console.WriteLine($"Perimetre : {2 * Math.PI * rayon:F2}");
Console.WriteLine($"Aire : {Math.PI * rayon * rayon:F2}");
```

> **Astuce DanielCraft** - `:F2` formate avec 2 decimales. Pratique pour l'affichage.

A retenir

- `$"..."` pour l'interpolation.
- `decimal` pour l'argent, `double` pour les calculs scientifiques.
- `:F2` pour formater les nombres.

Atelier : methodes



Atelier : creer des methodes.

Objectif

Creer des methodes avec differents types de retour et parametres.

Exercice 1 : salutation

```
string Saluer(string nom, int heure)
{
    if (heure < 12) return $"Bonjour {nom} !";
    if (heure < 18) return $"Bon apres-midi {nom} !";
    return $"Bonsoir {nom} !";
}

Console.WriteLine(Saluer("Lea", 9));
Console.WriteLine(Saluer("Max", 20));
```

Exercice 2 : moyenne

```
double Moyenne(int[] notes)
{
    if (notes.Length == 0) return 0;
    return notes.Average();
}

Console.WriteLine(Moyenne(new[] { 14, 16, 11, 18 }));
```

Exercice 3 : mot de passe valide

```
bool MdpValide(string mdp)
{
    return mdp.Length >= 8 && mdp.Any(char.IsDigit);
}

Console.WriteLine(MdpValide("abc"));           // False
Console.WriteLine(MdpValide("CSharp3!"));      // True
```

Exercice 4 : factorielle

```
long Factorielle(int n)
{
    if (n <= 1) return 1;
    return n * Factorielle(n - 1);
}

Console.WriteLine(Factorielle(5)); // 120
```

> **Astuce DanielCraft** - Teste chaque methode avec des cas normaux et des cas limites (0, vide, negatif).

A retenir

- Bien choisir le type de retour.
- Expression-bodied (`=>`) pour les methodes courtes.
- LINQ (`.Any()` , `.Average()`) simplifie le code.

Atelier : classes



Atelier : modeliser avec des classes.

Objectif

Modeliser des objets du monde reel avec des classes.

Exercice 1 : classe Produit

```
class Produit
{
    public string Nom { get; set; }
    public decimal Prix { get; set; }

    public Produit(string nom, decimal prix)
    {
        Nom = nom;
        Prix = prix;
    }

    public string Afficher() => $"{Nom} - {Prix:F2} EUR";
}

var p = new Produit("Clavier", 49.99m);
Console.WriteLine(p.Afficher());
```

Exercice 2 : panier d'achat

```
class Panier
{
    private List<Produit> _produits = new();

    public void Ajouter(Produit p) => _produits.Add(p);
    public decimal Total() => _produits.Sum(p => p.Prix);
    public void Afficher()
    {
        foreach (var p in _produits)
            Console.WriteLine($" {p.Afficher()}");
        Console.WriteLine($"Total : {Total():F2} EUR");
    }
}

var panier = new Panier();
panier.Ajouter(new Produit("Souris", 29.99m));
panier.Ajouter(new Produit("Ecran", 249.00m));
panier.Afficher();
```

Exercice 3 : heritage Vehicule

```
class Vehicule
{
    public string Marque { get; set; } = "";
    public virtual double Consommation() => 0;
}

class Voiture : Vehicule
{
    public override double Consommation() => 6.5;
}

class Moto : Vehicule
{
    public override double Consommation() => 4.0;
}
```

> **Astuce DanielCraft** - Pense "est-un" pour l'heritage. Une Voiture "est un" Vehicule. Un Panier "a des" Produits (composition).

A retenir

- Constructeur pour initialiser.
- `private` pour proteger les donnees internes.
- Heritage quand la relation "est un" est naturelle.

Erreurs classiques en C#



Pieges : null, index, iteration.

Les pieges des debutants

1. NullReferenceException

```
string? nom = null;
Console.WriteLine(nom.Length); // NullReferenceException
```

Verifie toujours si une variable peut etre null.

2. Oublier break dans switch

```
switch (x)
{
    case 1:
        Console.WriteLine("Un");
        // Oubli de break -> erreur de compilation en C#
        break;
}
```

En C#, le compilateur exige un `break` (pas de "fall-through" implicite).

3. Comparer des strings avec ==

En C#, `==` sur les strings compare les valeurs (contrairement a Java). Donc `==` fonctionne correctement. Piege evite !

4. Modifier une collection en iterant

```
foreach (var item in liste)
{
    liste.Remove(item); // InvalidOperationException !
}
```

Solution : creer une copie ou utiliser `RemoveAll` .

5. Index hors limites

```
int[] tab = { 1, 2, 3 };  
Console.WriteLine(tab[3]); // IndexOutOfRangeException
```

6. Oublier le point-virgule

```
Console.WriteLine("Oups") // Erreur CS1002
```

7. Confondre = et ==

```
if (x = 5) // Erreur : = assigne, == compare
```

Le compilateur C# refuse cette écriture (contrairement à C/C++).

> **Astuce DanielCraft** - Le compilateur C# est strict. La plupart des erreurs sont détectées avant même l'exécution.

A retenir

- Le compilateur est ton allié : lis ses messages.
- Null est le piège numéro 1.
- Ne modifie pas une collection pendant son itération.

Bonnes pratiques



PascalCase, petites methodes, analyseur.

Conventions de nommage

- Classes, methodes, proprietes : **PascalCase** .
- Variables locales, parametres : **camelCase** .
- Champs privés : **_camelCase** (prefixe underscore).
- Constantes : **PascalCase** (pas de UPPER_CASE en C#).

Structure d'un fichier

```
using System;
using System.Collections.Generic;

namespace MonApp;

public class MaClasse
{
    private readonly List<string> _items = new();

    public void AjouterItem(string item)
    {
        if (string.IsNullOrEmpty(item))
            throw new ArgumentException("Item vide");
        _items.Add(item);
    }
}
```

Principes

- **Une classe = une responsabilite.**
- **Methodes courtes** : si ca depasse 20 lignes, decoupe.
- **Immutabilite** : prefere **readonly** et **init** quand possible.
- **Nullable** : active les nullable reference types et gere les **?** .

Documentation

```
/// <summary>
/// Calcule le prix TTC a partir du HT.
/// </summary>
decimal CalculerTtc(decimal prixHt, decimal tva = 0.20m)
    => prixHt * (1 + tva);
```

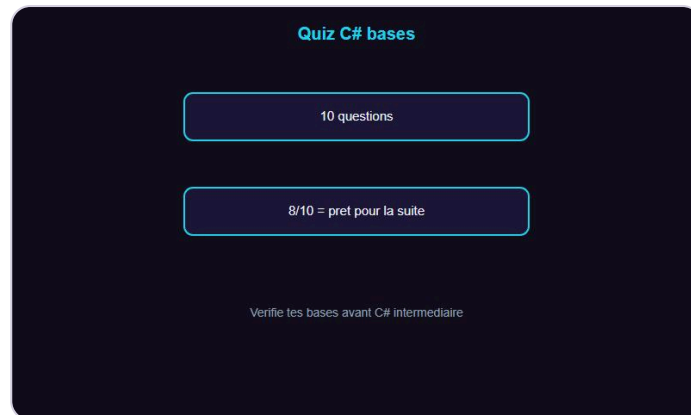
> **Astuce DanielCraft** - Installe un analyseur (Roslyn, SonarLint) pour detecter les problemes automatiquement.

A retenir

- PascalCase partout sauf variables locales.
- Le compilateur + analyseur = filet de securite.
- Petites methodes, noms explicites, tests.

QUIZ

Quiz C# - Les bases



Quiz C# bases.

Teste tes connaissances

Q1. Quel mot-cle declare une variable dont le type est deduit ?

A) auto B) var C) let D) dynamic

Q2. Quel type utiliser pour des calculs financiers precis ?

A) double B) float C) decimal D) int

Q3. Comment parcourir une liste sans index ?

A) for B) while C) foreach D) do-while

Q4. Quel est le resultat de `10 / 3` (deux int) ?

A) 3.33 B) 3 C) 4 D) Erreur

Q5. Comment lever une exception ?

A) raise B) throw C) except D) error

Q6. Quel modificateur empeche la modification d'une propriete de l'exterieur ?

A) readonly B) private set C) const D) static

Q7. Comment eviter un `NullReferenceException` ?

A) Ignorer B) Verifier null avant d'accéder C) Utiliser `dynamic` D) Desactiver les exceptions

Q8. Quel mot-cle permet a une classe enfant de redefinir une methode ?

A) new B) override C) replace D) redefine

Q9. `List<string>` est :

A) Un tableau fixe B) Une liste redimensionnable C) Un dictionnaire D) Une file

Q10. Quelle convention pour les noms de classes en C# ?

A) camelCase B) snake_case C) PascalCase D) UPPER_CASE

Reponses

1-B, 2-C, 3-C, 4-B, 5-B, 6-B, 7-B, 8-B, 9-B, 10-C

> **Astuce DanielCraft** - 8/10 ou plus ? Tu es pret pour C# intermediaire.

Bravo !

Bravo. Tu codes en C#.

Tu as termine C# - Les bases

Félicitations. Tu sais écrire des programmes C# types, structures et fonctionnels. Tu maîtrises les variables, les types, les conditions, les boucles, les méthodes, les collections, les classes, l'héritage et la gestion d'erreurs.

Ce que tu as construit

- Un premier programme console.
- Des méthodes claires et réutilisables.
- Un gestionnaire de tâches complet.
- Des exercices pratiques sur chaque notion.

La suite avec DanielCraft

Le livre **C# - Intermédiaire** couvrira :

- LINQ et les requêtes sur collections.
- Async/await.
- Les interfaces et l'injection de dépendances.
- Les génériques avancés.
- Introduction à ASP.NET et Unity.

> **Astuce DanielCraft** - Choisis un petit projet : un jeu console, un outil CLI, une API. Construis pour apprendre.

Un dernier conseil

C# est un langage qui grandit avec toi. Les bases te mènent aux applications professionnelles. Chaque projet terminé te rend meilleur.

Tu as les bases. Maintenant, construis.

Tu as les briques. Maintenant, construis.